



DATASOLUTION, CREATE A NEW WORLD WITH DATA

Prompt Engineering 실습

장준호



프롬프트 엔지니어링 기술 : 원칙과 전략

Prompt



Instruction
Context
Examples
Cue

원칙 1 : 명확하고 구체적인 지침

전략 1. 구분 기호를 사용하여 입력의 개별 부분을 명확하게 표시

전략 2. 구조화된 출력형식 요청

전략 3. 조건이 만족되는지 확인

전략 4. “N-shot” 프롬프팅

원칙 2 : 모델에게 "생각"할 시간을 주세요.

전략 1. 작업을 완료하는 데 필요한 단계 지정

전략 2. 성급하게 결론을 내리기 전에 모델에게 자체 솔루션을 마련하도록 지시

추가 전략 :

1. 단계적 생각 COT(Chain-of-Thought) : 다단계 문제에 대한 최종 답변을 제공하기 전에 중간 추론 단계를 생성합니다.

2. 생성된 지식 프롬프트 사용 : AI에게 주어진 질문/프롬프트에 대해 잠재적으로 유용한 정보를 생성하도록 요청한 다음 제공된 지식을 최종 질문/프롬프트 생성을 위한 추가 입력으로 활용하는 것입니다.

환경 설정

초기 설정

```
import os
from dotenv import load_dotenv          → 환경 변수들을 불러오는 함수
from ibm_watson_machine_learning.foundation_models import Model
from ibm_watson_machine_learning.metanames import GenTextParamsMetaNames as GenParams → 사용할 수 있는 파라미터 이름 목록
```

IBM Watson의 LLM 모델을 사용하기 위한 셋업

```
load_dotenv()
ibm_cloud_url = os.environ['https://us-south.ml.cloud.ibm.com']
project_id=os.environ['78753f00-619f-4a80-aa50-52242582d89f']
api_key = os.environ['rg_5Rx3rLg0YSK1rp4uYqXKJ9tebwbIa9cA7AwH4-aug']

creds = {
    "url": ibm_cloud_url,
    "apikey": api_key
}

# 모델 인스턴스 준비
model = Model(
    model_id = 'ibm/granite-3-3-8b-instruct',
    credentials=creds,
    project_id= project_id
)
```

환경 설정

LLM 모델에게 요청을 보내고 응답을 반환하는 함수 정의

```
# LLM 모델에게 요청을 보내고 응답을 반환하는 함수
def get_completion(prompt: str) -> str:
    response = model.generate(
        prompt=prompt,
        params={
            GenParams.MAX_NEW_TOKENS: 200,
            GenParams.TEMPERATURE: 0.7
        }
    )
    return response['results'][0]['generated_text']
```

1. 명확하고 구체적인 지침 작성 – S1 : Use Delimiters

입력 테스트의 범위를 명확하게 하기 위해 ****구분자(Delimiters)**** 를 사용합니다.

특히 긴 프롬프트나 여러 개의 입력 요소가 포함된 경우, 구분자를 통해 어디까지가 입력인지, 어떤 부분이 지시 사항인지를 명확히 구분할 수 있습니다.

****자주 사용하는 구분자 예시****:

- Triple quotes: `"""`
- Triple backticks: `````
- Triple dashes: `---`
- Angle brackets: `<>`
- XML tags: `\<tag>`

1. 명확하고 구체적인 지침 작성 – S1 : Use Delimiters

```
text = """
You should express what you want a model to do by providing instructions that are as clear and
specific as you can possibly make them.
This will guide the model towards the desired output, and reduce the chances of receiving
irrelevant or incorrect responses.
Don't confuse writing a clear prompt with writing a short prompt.
In many cases, longer prompts provide more clarity and context for the model, which can lead to
more detailed and relevant outputs.
"""

prompt = """
Summarize the text delimited by triple backticks into a single sentence. → 한문장으로 요약
```{text}```
"""

response = get_completion(prompt)
print(response)
```

# 1. 명확하고 구체적인 지침 작성 – S2 : 구조화된 결과 요청

모델의 출력을 JSON 등 구조화된 형태로 요구하면, 결과를 다른 프로그램이나 코드에서 바로 사용할 수 있습니다.

예를 들어, "JSON 형식으로 답변해줘"라고 지시하면 데이터 파싱, 후처리, API 응답 처리 등에 유리해집니다.

```
prompt = """
Generate a list of three made-up book titles along with their authors and genres.
Provide them in JSON format with the following keys: book_id, title, author, genre.
"""

response = get_completion(prompt)
print(response)
```

→ 가상의 책 제목 3개를 JSON형식으로 요청

# 1. 명확하고 구체적인 지침 작성 – S3 : 조건 충족 확인

모델에게 작업을 시작하기 전에 특정 조건이 충족되었는지를 먼저 확인하도록 지시할 수 있습니다. 조건이 충족되지 않을 경우, **\*\*작업을 중단하라\*\*** 는 지시도 함께 주면 불필요한 실행을 줄이고 출력 신뢰도와 제어 가능성을 높일 수 있습니다.

```
text_1 = """
Making a cup of tea is easy!
First, you need to get some water boiling.
While that's happening, grab a cup and put a tea bag in it.
Once the water is hot enough, just pour it over the tea bag.
Let it sit for a bit so the tea can steep.
After a few minutes, take out the tea bag.
If you like, you can add some sugar or milk to taste.
And that's it! You've got yourself a delicious cup of tea to enjoy.
"""

prompt = """
You will be provided with text delimited by triple quotes.
If it contains a sequence of instructions, re-write those instructions in the following format:
Step 1 - ...
Step 2 - ...
...
Step N - ...
If the text does not contain a sequence of instructions, then simply write "No steps provided."
\"\"\"{text_1}\"\"\"
"""

response = get_completion(prompt)
print("Completion for Text 1:")
print(response)
```

→ 텍스트에 명령 순서가 포함되어 있는 경우

→ 텍스트에 명령 순서가 포함되어 있지 않은 경우



# 1. 명확하고 구체적인 지침 작성 – S4 : N-Shot

모델에게 예시를 함께 제공해 예상 출력 형식과 스타일을 명확히 전달합니다.

유형	설명	예시
Zero-shot	예시 없이 바로 요청	"이 이메일이 스팸인가요?"
Few-shot	예시 2~5개를 함께 제공	번역 예시를 주고 요청 유도

```
prompt = """
Your task is to answer in a consistent style.
<child>: Teach me about patience.
<grandparent>: The river that carves the deepest valley flows from a modest spring; the grandest symphony originates
from a single note; the most intricate tapestry begins with a solitary thread.
<child>: Teach me about resilience.
"""
response = get_completion(prompt)
print(response)
```

→ 인내에 대한 질문

## 2. 모델이 생각할 시간 주기

복잡한 문제를 한 번에 해결하려 하기보다,

**\*\*단계를 나눠 순차적으로 해결하도록 유도\*\*** 하면 모델이 보다 정확하고 논리적인 결과를 도출할 수 있습니다.

이를 **\*\*Chain-of-Thought(사고의 연쇄)\*\*** 전략이라고 합니다.

## 2. 모델이 생각할 시간 주기 – S1:작업단계 명시

다음은 텍스트 요약, 프랑스어 번역, 프랑스어 요약본에 포함된 이름 목록 작성, JSON 형식으로 데이터를 출력하는 예시입니다.

필요한 단계를 제공함으로써 모델은 **\*\*이전 단계의 결과를 참조\*\*** 하고 출력의 정확성을 향상시킬 수 있습니다.

```
prompt = """
Your task is to perform the following actions:
1 - Summarize the following text delimited by <> with 1 sentence.
2 - Translate the summary into French.
3 - List each name in the French summary.
4 - Output a json object that contains the following keys: french_summary, num_names.

Use the following format:
Text: <text to summarize>
Summary: <summary>
Translation: <summary translation>
Names: <list of names in French summary>
Output JSON: <json with summary and num_names>

Text: <{text}>
"""

response = get_completion(prompt)
print(response)
```

## 2. 모델이 생각할 시간 주기 – S2:해결책을 제시

과제가 너무 복잡하거나 정보가 불충분할 경우, 모델은 추측을 통해 답을 내려고 할 가능성이 높습니다.

이런 경우, 모델에게 **\*\*먼저 문제를 분석하고, 스스로 논리를 전개한 뒤에 결론을 내리라고\*\*** 지시할 수 있습니다.

### 1) CoT(Chain-of-Thought) Prompting

2022년 Google 연구진이 소개한 방식으로, 복잡한 문제를 푸는 과정에서 중간 추론 단계를 명시적으로 생성하도록 유도하는 프롬프트 방식입니다.

이 방법은 모델이 다단계 문제를 중간 단계로 분해할 수 있도록 하여, 표준 프롬프팅 방식으로는 해결할 수 없는 복잡한 추론 문제를 해결할 수 있게 합니다.

CoT 유형별 전략 및 예시

Type	Description	Example
Self-consistency prompting	모델이 여러 개의 다양한 추론 경로를 생성한 후, 가장 일관성 있는 답변을 선택하게 함	예: 날씨 예측 시 "비", "흐림", "맑음" 등 다양한 답변을 생성한 뒤, 가장 자주 등장하거나 타당한 결과를 선택
Least-to-Most prompting (LtM)	복잡한 문제를 먼저 단계별 하위 문제로 쪼개고, 순차적으로 해결한 뒤 최종 답을 도출함	예: 복잡한 수학 문제를 작게 나눈 다음, 각 부분을 해결하고 결과를 합쳐 최종 답 도출
Active Prompting	모델이 먼저 불확실하거나 핵심적인 부분을 파악하고, 해당 부분에 대해 사용자의 보완 입력(피드백/주석 등)을 요청한 후 이를 반영	예: 리포트 작성 시 불확실한 수치나 주장 부분을 표시해 사용자 피드백을 요청하고, 반영하여 완성도 향상

## 2. 모델이 생각할 시간 주기 – S2:해결책을 제시

### 2) Generated Knowledge Prompting

모델에게 최종 작업을 바로 수행하도록 지시하는 대신, 먼저 관련된 배경 지식이나 핵심 정보를 생성하도록 유도한 뒤, 이를 활용해 최종 출력을 생성하게 하는 전략입니다.

예를 들어, 사이버 보안에 대한 블로그 글을 작성하기 전에 "쿠키 탈취의 위험 요소와 방지 방법"에 대한 정보를 먼저 생성하게 한 다음, 해당 내용을 바탕으로 본문을 작성하도록 하면 더 풍부하고 유익한 결과물을 얻을 수 있습니다.

### 3) Prompt Tuning

Prompt Tuning은 기존의 텍스트 기반 프롬프트 대신, "소프트 프롬프트"라 불리는 학습 가능한 임베딩 벡터를 사용해 언어 모델을 미세 조정하는 기법입니다.

GPT-3에서 일반적으로 사용되는 이산적 텍스트 프롬프트와 달리, 소프트 프롬프트는 역전파(backpropagation) 과정을 통해 훈련되며, 레이블이 지정된 소수의 예시로부터 학습 신호를 통합할 수 있도록 조정됩니다.

이 방법은 GPT-3의 few-shot learning보다 더 높은 성능을 보이는 경우가 많으며, 전체 모델 파라미터를 조정하지 않아도 되기 때문에 효율성과 확장성 측면에서 매우 유리합니다.



DATASOLUTION, CREATE A NEW WORLD WITH DATA

# THANK YOU

